

Demiryolu & Metro Sistemleri RAG Projesi

Ayrıntılı Veri Seti Analizi, Vektör Veritabanı Mimarisi, Metin Anlama ve Değerlendirme Raporu

Gelişmiş & Detaylandırılmış Teknik Sürüm (v3.0)

Projenin Amacı ve "Metin Anlama" Mantiğı:

Projenin temel amacı, kullanıcının girdiği "tren", "deray", "çatlak ray", "makas arızası", "metro gecikmesi" veya "vagon devrilmesi" gibi anahtar kelimeleri ve karmaşık soruları yalnızca kelime eşleşmesiyle (keyword search) değil, **Anlamsal Arama (Semantic Search)** ile anlayıp verilen 3 heterojen veri setinden en doğru bağlamı (context) çekmek ve LLM aracılığıyla doğrulanmış yanıtlar üretmektir.

1. Veri Kaynaklarının Detaylı ve Teknik İncelemesi

1.1. Railway Track Fault Detection (Kaggle)

Erişim Adresi: <https://www.kaggle.com/datasets/salmaneunus/railway-track-fault-detection>

- Veri Türü ve Yapısı:** Ray üzerindeki fiziksel ve teknik arızaları (çatlak, kırık, gevşek bağlantı elemanı, hat hizalama bozukluğu) tespit etmeye yönelik görüntü ve metinsel etiket verileri.
- İşlenecek Sütunlar ve Özellikler:** Fault Type (Arıza Türü), Location ID (Hat/Ray Kimliği), Severity Level (Hasar Şiddet Derecesi), Inspection Method (Denetim Yöntemi).
- RAG Projesindeki Rolü:** Kullanıcı "tren rayındaki teknik sorunlar nelerdir?" veya "ray çatlakları nasıl tespit edilir?" diye sorduğunda RAG sistemi bu verideki teknik arıza kategorilerini bağlam olarak çekerek LLM'e sunar.

1.2. Rail Equipment Accidents - U.S. Department of Transportation (DOT)

Erişim Adresi: <https://data.transportation.gov/Railroads/Rail-Equipment-Accidents/xdpv-m434>

- Veri Türü ve Yapısı:** Resmi demiryolu kazaları, ekipman hasarları, deray (raydan çıkma) olayları ve detaylı kaza anlatıları (Narratives) içeren çok geniş tabular ve metinsel veri seti.
- Kritik Özellikler (Fields):**
 - Narrative: Kaza oluş şeklini anlatan doğal dil özetleri (RAG metin anlama için en kritik sütun).
 - Accident Cause / Cause Code: Kaza nedeni (Mekanik arıza, insan hatası, ray bozukluğu, sinyalizasyon hatası vb.).
 - Train Speed & Track Class: Tren hızı ve hat güvenlik sınıfı.
 - Equipment Damage Cost: Hasar ve ekipman maliyet tutarları.
- RAG Projesindeki Rolü:** Metinsel Narrative sütunu, embedding modelinden geçirilerek vektörleştirilir. Kullanıcı "tren kazalarında en sık karşılaşılan mekanik nedenler nelerdir?" veya "yüksek hızda deray olma nedenleri" dediğinde bu veri setinden gerçek kaza vakaları çekilir.

1.3. Metro Train Dataset (Kaggle)

Erişim Adresi: <https://www.kaggle.com/datasets/anshtanwar/metro-train-dataset>

- Veri Türü ve Yapısı:** Şehir içi metro hatları, istasyonlar, tren sefer süreleri, gecikme süreleri ve yolcu yoğunluğu verileri.
- İşlenecek Sütunlar ve Özellikler:** Station Name (İstasyon Adı), Schedule / Delay Minutes (Sefer/Gecikme Bilgisi), Passenger Volume (Yolcu Yoğunluğu), Line ID (Hat Bilgisi).
- RAG Projesindeki Rolü:** "Şehir içi metro trenlerinde gecikmeye yol açan etkenler nelerdir?" veya "belirli hatlardaki sefer yoğunluğu" gibi metro odaklı kullanıcı sorgularını yanıtlamak için kullanılır.

2. Veri Ön İşleme (Pre-processing) & Metin Temizleme Adımları

- Metin Normalizasyonu & Temizliği:** Kaza özetlerindeki (Narratives) HTML etiketleri, özel karakterler, çift boşluklar temizlenir. Tüm metin standart küçük harfe dönüştürülür.
- Tabular Verilerin Metinleştirilmesi (Table-to-Text):** Tablo sütunları (örn: Speed: 45 mph, Cause: Rail Fracture) "Tren 45 mph hızla ilerlerken ray kırılması sebebiyle kaza yaptı" şeklinde doğal dil cümlelerine dönüştürülür.

3. **Akıllı Parçalama (Overlapping Chunking):** Metinler 300-500 token'lık parçalara bölünür. Cümle bütünlüğünü kaybetmemek için %10-15 (50 token) örtüşme (overlap) uygulanır.
4. **Metadata Etiketleme:** Her parçaya `source_file`, `accident_year`, `cause_category` gibi metadata etiketleri eklenerek filtreli arama altyapısı kurulur.

3. Metin Anlayabilme (Semantic Search) & Vektör Veritabanı Mimarisi

Mimari Katman	İşlem Mantığı ve Teknik Detay	Seçilen Teknolojiler
1. Chunking (Parçalama)	Kaza anlatıları ve arıza tanımları 300-500 token'lık metin parçalarına bölünür. 50 token örtüşme ile anlam korunur.	LangChain / LlamaIndex RecursiveCharacterTextSplitter
2. Embedding (Vektörleştirme)	Metinler 1536 boyutlu sayısal vektörlere dönüştürülür. "Tren", "Lokomotif", "Vagon", "Katar" kelimeleri anlamsal uzayda yakınlaşır.	OpenAI text-embedding-3-small veya HuggingFace BGE-Large-EN
3. Vector DB Depolama	Vektörler ve karşılık gelen ham metinler (metadata ile) HNSW indeksi kullanılarak kaydedilir.	Qdrant / ChromaDB / Pinecone / FAISS
4. Retrieval (Arama & Çekme)	Kullanıcı sorusu da vektörleştirilir. Kosinüs Benzerliği (Cosine Similarity) ile en alakalı top-10 parça çekilir ve Cross-Encoder ile re-rank edilir.	Cosine Similarity + Cohere Rerank v3
5. LLM Yanıt Üretimi	Çekilen en alakalı top-3 bağlam isteme (prompt) eklenerek uydurmasız (hallucination-free) teknik yanıt üretilir.	GPT-4o / Claude 3.5 Sonnet / Llama 3

4. Gelişmiş RAG Python Uygulama Kodu

```
# DEMIR YOLU VE METRO RAG PIPELINE - FULL IMPLEMENTATION
from langchain_community.vectorstores import Qdrant
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain.chains import RetrievalQA
from langchain.prompts import PromptTemplate

# 1. Embedding ve Vektör Veritabanı Bağlantısı
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vector_db = Qdrant.from_existing_collection(
    embedding=embeddings,
    collection_name="railway_metro_rag_db",
    url="http://localhost:6333"
)

# 2. Retriever ve Re-ranker Yapılandırması
retriever = vector_db.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 5}
)

# 3. Özel RAG Sistem Prompt Şablonu
prompt_template = """
Sen bir Demiryolu, Tren ve Metro Sistemleri Teknik Uzmanısın.
Sana verilen BAGLAM metinlerini kullanarak kullanıcının sorusunu yanıtla.
Baglamda cevabi bulunmayan sorular için uydurma yanıt verme, "Verilen verilerde bu sorunun cevabi bulunmamaktadır" de.

BAGLAM METINLERİ:
{context}

KULLANICI SORUSU:
{question}

TEKNİK VE ANLAMSAL YANIT:
"""

PROMPT = PromptTemplate(template=prompt_template, input_variables=["context", "question"])

# 4. LLM ve QA Chain Kurulumu
llm = ChatOpenAI(model_name="gpt-4o", temperature=0.1)
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    chain_type="stuff",
    retriever=retriever,
    chain_type_kwargs={"prompt": PROMPT}
)

# 5. Örnek Sorgu Çalıştırma
user_query = "Ray catlagi kaynakli deray kazalarının ana nedenleri ve tren hizinin buna etkisi nedir?"
response = qa_chain.run(user_query)
print("RAG Yaniti:\n", response)
```

5. RAG Başarım Ölçümü ve Değerlendirme Metrikleri (RAGAS)

- **Faithfulness (Sadakat - >0.90):** Üretilen yanıtın tamamen verilen bağlama sadık kalıp kalmadığını ölçer (Hallucination kontrolü).
- **Answer Relevance (Yanıt Alakası - >0.85):** Üretilen yanıtın kullanıcının sorduğu soru ile ne kadar doğrudan alakalı olduğunu ölçer.
- **Context Recall (Bağlam Kapsamı - >0.88):** Vektör veritabanından çekilen parçaların, soruyu yanıtlamak için gereken tüm bilgileri kapsayıp kapsamadığını ölçer.
- **Context Precision (Bağlam Hassasiyeti - >0.90):** Çekilen parçalar içindeki gürültü (gereksiz metin) oranını ölçer. High Precision = Temiz Bağlam.

6. Proje Ekibi İçin Görev Dağılımı ve İş Paketi Önerisi

1. **Veri Hazırlık Sorumlusu:** U.S. DOT veri setindeki Narrative sütunu ile Kaggle'daki arıza/metro metinlerini temizleyip JSON/CSV formatına getirecek (1. Hafta).
2. **Vektör Mimarisi Sorumlusu:** LangChain veya LlamaIndex kullanarak metin parçalama (chunking) ve ChromaDB/Qdrant vektör veritabanı kurulumunu yapacak (2. Hafta).
3. **LLM & Prompt Mühendisi:** Retrieval-QA zincirlerini kuracak ve RAGAS metrikleri ile başarımlarını testlerini gerçekleştirecek (3. Hafta).
4. **Arayüz ve Dağıtım Sorumlusu:** Streamlit veya Gradio ile kullanıcı paneli hazırlayıp LLM çıktıları ekranda gösterecek (4. Hafta).